Lead Scoring Prédictif

Voici un guide méthodologique complet et opérationnel pour concevoir, implémenter et soutenir ce projet de **Lead Scoring Prédictif** dans le cadre du module **Web Marketing & CRM**, en mobilisant les leviers de l'intelligence artificielle et de la transformation digitale.

1. Démarche d'Implémentation :

La démarche pour réaliser ce prototype fonctionnel s'articule autour des étapes suivantes :

⇒ **Acquisition des Données :** Il faut générer ou utiliser un jeu de données de leads contenant des variables marketing telles que: ***la source***, ***le secteur***, ***les pages vues*** et ***les e-mails ouverts***;

⇒ **Modélisation IA :** L'objectif est d'entraîner un modèle de machine learning capable de ***prédire la probabilité de conversion d'un lead***. Cette approche prédictive basée sur l'intelligence artificielle doit ensuite être ***comparée au scoring manuel traditionnel***;

⇒ **Intégration CRM :** Une fois les scores calculés, ils doivent être ***intégrés dans Odoo en utilisant un fichier d'import*** pour mettre à jour le champ de probabilité ou la priorité des opportunités.

2. Choix et Justification du Second CRM à coté d'Odoo CRM (avec des limites 2026):

✓ **HubSpot CRM:** En 2026, l'offre gratuite en ligne est plafonnée à 2 utilisateurs et 1 000 contacts. Elle inclut la marque HubSpot sur les e-mails et le chat, et limite l'analyse à 10 tableaux de bord. Toutefois, son orientation native vers l'inbound marketing (tracking de l'engagement) le rend idéal pour capturer et illustrer les variables comportementales (pages vues, e-mails ouverts) nécessaires à l'entraînement du modèle d'IA. Le plafond de 1 000 contacts est par ailleurs largement suffisant pour importer un jeu de données test et réaliser un prototype fonctionnel.

3. Livrables Attendus pour la Démonstration:

- **Notebook :** Le code informatique (généralement Python) détaillant ***l'exploration des données et l'entraînement du modèle***;
- **Courbe ROC :** Une visualisation graphique de ***la performance du modèle***;
- **Fichier d'import :** Le document structure (est le format **CSV** ou, à défaut, le format **Excel (.xlsx)**) contenant les leads et leurs scores prédictifs calculés;
- **Démonstration :** Une présentation en direct dans Odoo montrant le pipeline de ventes trié par le score.

4. Jeu de Données - Source et Variables Marketing:

Pour rendre le prototype réaliste, on peut exploiter un jeu de données public standard du domaine : le dataset Kaggle **« Lead Scoring Dataset »** (issu d'une plateforme d'e-learning / EdTech. Il comprend 9 240 leads et 37 attributs.

Le lien à Dataset (Kaggle): https://www.kaggle.com/datasets/amritachatterjee09/lead-scoring-dataset.

Variables recommandées pour entraîner le modèle (Features web marketing):

- `Variable cible (Target) :` 'Converted' (0 = lead perdu, 1 = lead converti).
- `Canaux d'acquisition :` 'Lead Origin' (origine de la piste : Landing Page Submission, API, Lead Add Form) et 'Lead Source' (Google, Direct Traffic, Organic Search, Referral Sites, etc.).
- `Métriques comportementales web :` 'TotalVisits' (nombre de visites), 'Total Time Spent on Website' (temps total passé sur le site en secondes) et 'Page Views Per Visit' (pages vues par session).
- `Engagement et nurturing :` 'Last Activity' (dernière interaction : Email Opened, SMS Sent, Page Visited) et 'Do Not Email' (opt-out emailing).
- `Profil et secteur :` 'Specialization' (secteur professionnel ou domaine d'études) et 'What is your current occupation' (statut : salarié, demandeur d'emploi, étudiant).

5. Implémentation dans le Notebook Jupyter / Colab:

Le notebook se structure en 4 blocs obligatoires:

- **Étape A : Prétraitement et Nettoyage**

```
import numpy as np
import pandas as pd

# ------------------------------------------------------------------------
# ÉTAPE 1 : Chargement et conversion des faux "Select" en NaN
# ------------------------------------------------------------------------
df = pd.read_csv("Lead Scoring.csv")

df = df.replace("Select", np.nan)

# ------------------------------------------------------------------------
# ÉTAPE 2 : Suppression des identifiants et des colonnes "Data Leakage"
# ------------------------------------------------------------------------
ids_to_drop = ["Prospect ID", "Lead Number"]

# 2. Variables remplies par les équipes commerciales APRÈS contact (Data Leakage)
sales_leakage_cols = [
    "Tags",
    "Lead Quality",
    "Lead Profile",
```

```
    "Asymmetrique Activity Index",
    "Asymmetrique Profile Index",
    "Asymmetrique Activity Score",
    "Asymmetrique Profile Score",
    "Last Notable Activity",  # Redondant avec 'Last Activity' et souvent mis à jour
manuellement
]

df.drop(columns=ids_to_drop + sales_leakage_cols, inplace=True, errors="ignore")

# ------------------------------------------------------------------------
# ÉTAPE 3 : Suppression des colonnes trop creuses (> 40% de valeurs manquantes)
# ------------------------------------------------------------------------
# Calcul du pourcentage de valeurs manquantes
missing_percentage = (df.isnull().sum() / len(df)) * 100
cols_high_missing = missing_percentage[missing_percentage > 40].index.tolist()

# Note : On conserve 'Specialization' si elle est dans la liste, car un lead qui
# ne précise pas son secteur est une information métier pertinente (Non renseigné)
if "Specialization" in cols_high_missing:
  cols_high_missing.remove("Specialization")

df.drop(columns=cols_high_missing, inplace=True)

# ------------------------------------------------------------------------
# ÉTAPE 4 : Suppression des colonnes constantes ou univariées
# ------------------------------------------------------------------------
skewed_cols = [
    col
    for col in df.columns
    if df[col].value_counts(normalize=True, dropna=False).values[0] > 0.98
    and col != "Converted"
]
df.drop(columns=skewed_cols, inplace=True)

# ------------------------------------------------------------------------
# ÉTAPE 5 : Standardisation et regroupement des modalités (Web Marketing)
# ------------------------------------------------------------------------
# A. Nettoyage de 'Lead Source'
if "Lead Source" in df.columns:
  # Harmonisation de la casse
  df["Lead Source"] = df["Lead Source"].replace({"google": "Google"})

  # Regroupement des sources à très faible volume sous 'Other_Source'
  top_sources = [
      "Google",
      "Direct Traffic",
      "Organic Search",
      "Olark Chat",
      "Reference",
  ]
  df["Lead Source"] = df["Lead Source"].apply(
      lambda x: x if x in top_sources else ("Other_Source" if pd.notnull(x) else x)
  )
```

```
  df["Lead Source"].fillna("Other_Source", inplace=True)

# B. Traitement des variables de profil (Spécialisation et Métier)
if "Specialization" in df.columns:
  df["Specialization"].fillna("Not_Specified", inplace=True)

if "What is your current occupation" in df.columns:
  df["What is your current occupation"].fillna("Unemployed", inplace=True)

if "Last Activity" in df.columns:
  top_activities = [
      "Email Opened",
      "SMS Sent",
      "Page Visited on Website",
      "Olark Chat Conversation",
  ]
  df["Last Activity"] = df["Last Activity"].apply(
      lambda x: (
          x if x in top_activities else ("Other_Activity" if pd.notnull(x) else x)
      )
  )
  df["Last Activity"].fillna("Other_Activity", inplace=True)

# ---------------------------------------------------------------------------
# ÉTAPE 6 : Traitement des variables numériques & Outliers (Écrêtage)
# ---------------------------------------------------------------------------
num_cols = ["TotalVisits", "Total Time Spent on Website", "Page Views Per Visit"]
num_cols = [c for c in num_cols if c in df.columns]

# Imputation par la médiane pour les variables web
for col in num_cols:
  median_val = df[col].median()
  df[col].fillna(median_val, inplace=True)

# Plafonnement des valeurs extrêmes (Outliers) au 99ème percentile
# (Ex : un robot qui visite 200 pages en une session fausserait les coefficients)
for col in ["TotalVisits", "Page Views Per Visit"]:
  if col in df.columns:
    q_99 = df[col].quantile(0.99)
    df[col] = np.where(df[col] > q_99, q_99, df[col])

# ---------------------------------------------------------------------------
# ÉTAPE 7 : Encodage des variables binaires (Yes / No)
# ---------------------------------------------------------------------------
binary_cols = ["Do Not Email", "Do Not Call"]
for col in binary_cols:
  if col in df.columns:
    df[col] = df[col].map({"Yes": 1, "No": 0}).fillna(0).astype(int)

# ---------------------------------------------------------------------------
# DIAGNOSTIC FINAL APRÈS NETTOYAGE
# ---------------------------------------------------------------------------
print(f"Dimensions du dataset final prêt à modéliser : {df.shape}")
print(
```

```
    f"Nombre de valeurs manquantes restantes : {df.isnull().sum().sum()} (doit être 0)"
)
print("\nDistribution de la variable cible 'Converted' :")
print(df["Converted"].value_counts(normalize=True).round(3))
```

- **Étape 2 : Modélisation du Scoring Manuel (Benchmark Métier)**

Le sujet exige de **comparer le modèle IA au scoring manuel**. On crée une fonction qui simule la grille de points arbitraire qu'un responsable marketing utilise habituellement :

```
def manual_lead_score(row):
  score = 0
  # Règles arbitraires basées sur l'intuition marketing classique
  if row.get("Lead Source") in ["Google", "Reference"]:
    score += 25
  if row.get("Total Time Spent on Website", 0) > 400:
    score += 35
  elif row.get("Total Time Spent on Website", 0) > 150:
    score += 15
  if row.get("Last Activity") in ["Email Opened", "SMS Sent"]:
    score += 25
  if row.get("TotalVisits", 0) >= 3:
    score += 15

  # Normalisation entre 0 et 1 (probabilité équivalente)
  return min(score, 100) / 100.0


# Calcul du score manuel sur le jeu de test
y_scores_manual = X_test.apply(manual_lead_score, axis=1)
```

- **Étape 3 : Entraînement du Modèle IA Prédictif**

Le modèle le plus recommandé est sans hésitation la ***Régression Logistique régularisée*** **(intégrée dans un Pipeline Scikit-Learn)**, avec possibilité de la benchmarker face à une ***Forêt Aléatoire (*Random Forest*)***.

⇒ **Pourquoi la Régression Logistique est le meilleur choix en CRM ?**

1. **Probabilités natives et parfaitement calibrées :**
   Contrairement aux modèles ensemblistes (Random Forest, XGBoost) dont les scores de sortie représentent des fractions de votes d'arbres (qui s'agglutinent souvent entre 0.3 et 0.7 et nécessitent un recalibrage complexe), la régression logistique modélise directement la fonction sigmoïde :

$$P(Y = 1 \mid X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p)}}$$

   Ce résultat correspond exactement à ce qu'attend le champ natif **probability** d'Odoo (0 à 100%).
2. **Explicabilité commerciale totale (*Explainable AI / XAI*) :**
   Dans une démarche CRM, une équipe commerciale rejette les modèles « boîte noire ». Avec la

régression logistique, chaque coefficient β_i se traduit en **Odds Ratio** (e^{β_i}). *Exemple : « Si le prospect a reçu un SMS, ses chances de convertir sont multipliées par 2,4 ».*

3. **Robustesse et rapidité de déploiement :**
   Sur des données tabulaires marketing de quelques milliers de lignes, elle évite le surapprentissage (*overfitting*) et s'exécute instantanément.

- **Étape 4 : Évaluation comparative et Courbe ROC (Livrable attendu)**

Cette partie ne se contente pas de tracer le graphique : elle interprète les résultats sous le prisme du **ROI commercial** et du **Web Marketing** (coût du temps passé par les commerciaux, taux de capture des leads chauds).

6. Préparation et Génération des Fichiers (Livrable attendu):

7. Démonstration Pratique pas-à-pas dans les CRM (Odoo & HubSpot) (Livrable attendu):

- **Étape 1 : Préparation du fichier CSV pour HubSpot (Code Python)**

Exécutez ce court script à la suite de votre notebook pour générer import_hubspot_leads.csv :

```
import pandas as pd
import numpy as np

# 1. Récupération des données test scorées par l'IA et le manuel
hubspot_df = pd.DataFrame({
    # Identifiant unique obligatoire pour HubSpot
    'Email': [f"prospect_{idx}@entreprise{idx}.ma" for idx in X_test.index],
    'First Name': [f"Lead_{idx}" for idx in X_test.index],
    'Company Name': [f"Société_{idx} SARL" for idx in X_test.index],

    # Données Web Marketing
    'Lead Source': X_test['Lead Source'].values if 'Lead Source' in X_test.columns else
'Web Inbound',

    # Livrables du modèle : Scores IA et Manuel
    'Lead Score IA': np.round(y_scores_ia * 100, 1),       # Probabilité continue (ex:
88.5)
    'Lead Score Manuel': np.round(y_scores_manual * 100, 1), # Score heuristique (ex:
60.0)

    # Statut d'acquisition
    'Lifecycle Stage': 'Marketing Qualified Lead'
})

# 2. Exportation du fichier propre
hubspot_df.to_csv("import_hubspot_leads.csv", index=False, encoding="utf-8")
```

```
print(f"☑ Fichier prêt pour HubSpot exporté : {hubspot_df.shape[0]} contacts générés.")
display(hubspot_df.head(3))
```

- **Étape 2 : Configuration préalable dans HubSpot CRM (Gratuit)**

Avant d'importer le fichier, vous devez créer le ***champ personnalisé dans HubSpot*** pour accueillir la probabilité issue de l'IA (ce qui consomme 1 propriété sur les 10 autorisées par l'offre gratuite) :

1. Connectez-vous sur votre compte **HubSpot Free**.
2. Cliquez sur l'icône **Paramètres** (en haut à droite).
3. Dans le menu de gauche, allez dans **Gestion des données → Propriétés**.
4. Sélectionnez l'objet **Propriétés du contact**, puis cliquez sur le bouton orange **Créer une propriété** :
    - **Groupe :** Informations sur le contact
    - **Libellé (Label) :** Lead Score IA
    - **Type de champ :** Nombre (Sélectionnez format *Nombre sans mise en forme* ou *Pourcentage*)
5. Cliquez sur **Créer**. *(Répétez l'opération pour Lead Score Manuel si vous souhaitez comparer les deux colonnes côte à côte)*.

- **Étape 3 : Importation des données dans HubSpot (Procédure démo)**

- Dans le menu principal HubSpot, allez dans **CRM → Contacts**.
- Cliquez sur le bouton **Importer** (en haut à droite) → **Démarrer un import**.
- Choisissez : **Fichier depuis un ordinateur → Un seul fichier → Un seul objet** → Sélectionnez **Contacts**.
- Téléversez votre fichier import_hubspot_leads.csv.
- **Vérification du Mappage (Mapping) :**
    - Email → mappé automatiquement à *E-mail*
    - First Name → mappé à *Prénom*
    - Company Name → mappé à *Nom de l'entreprise*
    - Lead Score IA → reliez-le à votre propriété personnalisée **Lead Score IA**
    - Lead Score Manuel → reliez-le à **Lead Score Manuel**
- Cochez la case de consentement légal, donnez un nom à l'import (ex. : *Test Scoring IA 2026*), puis cliquez sur **Finaliser l'importation**.
- **Étape 4 : Mise en valeur pour la Démonstration (Vue Kanban / Liste triée)**

Pour montrer la priorisation commerciale en direct :

1. Retournez dans **CRM → Contacts**.
2. Cliquez sur **Personnaliser les colonnes** (à droite du tableau).
3. Cochez **Lead Score IA** pour l'ajouter à l'affichage et placez-la au début du tableau.

4. Cliquez sur l'en-tête de la colonne **Lead Score IA** pour effectuer un **tri décroissant (de 100% à 0%)**.
5. **Résultat visible :** Le commercial dispose immédiatement de sa liste d'appels ultra-ciblée, avec les prospects à fort potentiel en tête de liste.